



C'est parti ! Il faut faire évoluer l'application de ToDo & Co

ToDo & Co

Documentation technique

Guide d'authentification

Sommaire

Sommaire

Introduction.....	3
Les utilisateurs.....	4
Système d'authentification.....	5
Les rôles.....	6
Le contrôle des accès.....	7
access_control.....	7
.....	7
Attributes.....	7

Introduction

L'application ToDo&Co est développée sur la base du Framework Symfony qui propose un composant performant permettant de gérer l'authentification des utilisateurs : le composant Security. Ce composant permet d'une part de gérer l'authentification, à savoir d'où proviennent les utilisateurs et comment gérer l'authentification.

D'autre part, ce système permet également de gérer un système d'autorisations, à savoir gérer l'accès à certaines ressources selon le rôle défini de l'utilisateur (visiteur non authentifié, utilisateur authentifié ou encore administrateur).

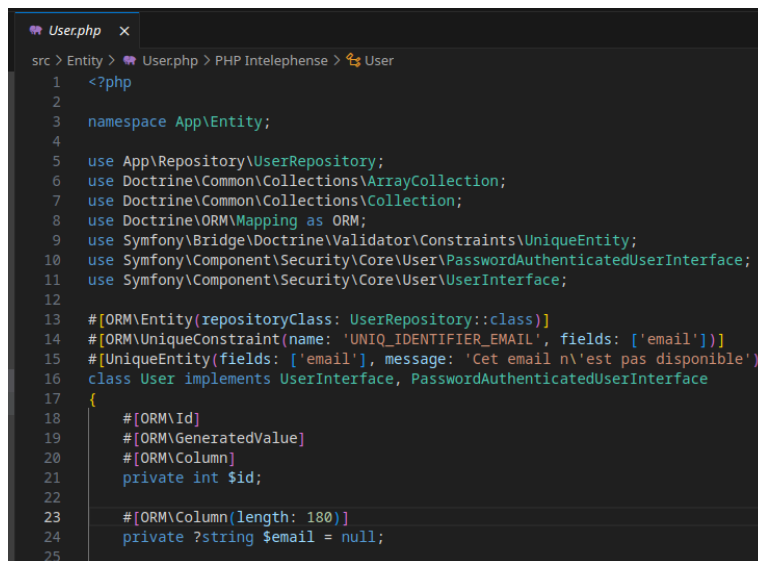
Ces deux aspects seront traités dans les deux parties suivantes. La majorité des configurations pour ces différents volets sont regroupées au sein du fichier config/packages/security.yaml.

Ce guide a pour objectif d'explicitier le système d'authentification mis en place pour l'application ToDo&Co, afin d'assurer une correcte compréhension par tous les développeurs qui contribueront au développement de l'application, et de définir quels fichiers peuvent être modifiés et comment afin de pallier aux évolutions de l'application.

Authentication

Les utilisateurs

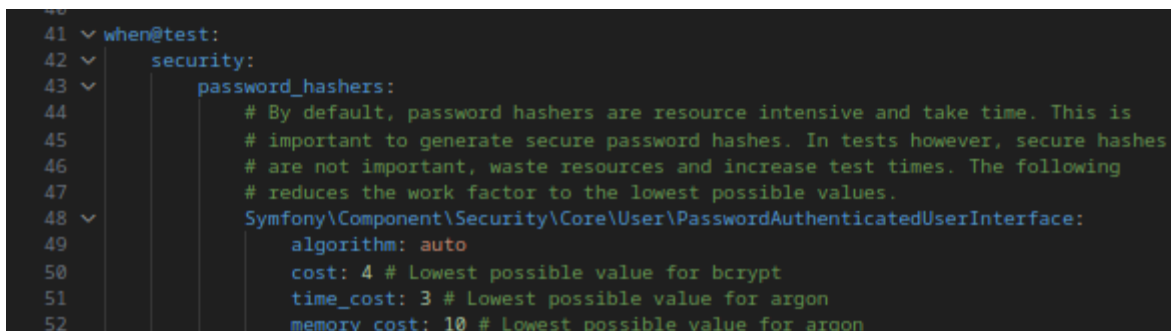
- La configuration des utilisateurs est gérée au sein du fichier *security.yaml*.
- L'entité *User* représente l'utilisateur. C'est une entité Doctrine.
- Les utilisateurs sont stockés dans la base de données.
- Un utilisateur est identifié par son *email* qui est unique dans la base de données.



```
1 <?php
2
3 namespace App\Entity;
4
5 use App\Repository\UserRepository;
6 use Doctrine\Common\Collections\ArrayCollection;
7 use Doctrine\Common\Collections\Collection;
8 use Doctrine\ORM\Mapping as ORM;
9 use Symfony\Bridge\Doctrine\Validator\Constraints\UniqueEntity;
10 use Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface;
11 use Symfony\Component\Security\Core\User\UserInterface;
12
13 #[ORM\Entity(repositoryClass: UserRepository::class)]
14 #[ORM\UniqueConstraint(name: 'UNIQ_IDENTIFIER_EMAIL', fields: ['email'])]
15 #[UniqueEntity(fields: ['email'], message: 'Cet email n\'est pas disponible')]
16 class User implements UserInterface, PasswordAuthenticatedUserInterface
17 {
18     #[ORM\Id]
19     #[ORM\GeneratedValue]
20     #[ORM\Column]
21     private int $id;
22
23     #[ORM\Column(length: 180)]
24     private ?string $email = null;
25 }
```

- La classe *EmailVerifier*, permet de valider la création d'un utilisateur et de confirmer son mail.

L'authentification est réalisée sur la base de l'adresse *email* de l'utilisateur et d'un mot de passe. Le mot de passe ne doit pas être enregistré en clair dans la base de données. Le paramètre *password_hashers* permet d'assurer un encodage du mot de passe avant l'enregistrement de l'utilisateur. L'algorithme de hachage se trouve dans le fichier *security.yaml*.



```
41 when@test:
42     security:
43         password_hashers:
44             # By default, password hashers are resource intensive and take time. This is
45             # important to generate secure password hashes. In tests however, secure hashes
46             # are not important, waste resources and increase test times. The following
47             # reduces the work factor to the lowest possible values.
48             Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
49                 algorithm: auto
50                 cost: 4 # Lowest possible value for bcrypt
51                 time_cost: 3 # Lowest possible value for argon
52                 memory_cost: 10 # Lowest possible value for argon
```

Système d'authentification

Le système d'authentification du composant « Security » repose sur la définition du **firewalls**. Il permet de déterminer comment les utilisateurs sont authentifiés, et notamment via **firewalls --main**.

```
12  ✓  firewalls:
13  ✓      dev:
14          pattern: ^/(_(profiler|wdt)|css|images|js)/
15          security: false
16  ✓      main:
17          lazy: true
18          provider: app_user_provider
19  ✓      form_login:
20          login_path: app_login
21          check_path: app_login
22          enable_csrf: true
23  ✓      logout:
24          path: app_logout
25          # where to redirect after logout
26          # target: app_any_route
27
28          # activate different ways to authenticate
29          # https://symfony.com/doc/current/security.html#the-firewall
30
31          # https://symfony.com/doc/current/security/impersonating\_user.html
32          # switch_user: true
```

- *Lazy:true* permet d'éviter de créer une nouvelle session lorsqu'il n'y a pas besoin d'autorisation particulière. Cela permet la mise en cache de toutes les URLs ne nécessitant pas d'utilisateur.
- L'authentification mise en place, passe par le formulaire de connexion, la route d'accès au formulaire de login est dans le *login_path*. Ce formulaire est protégé par un jeton *CSFR*.
- Le paramètre de *logout* permet de définir la route de déconnexion des utilisateurs.

Les rôles

Par l'implémentation de l'interface *UserInterface*, la classe **User** possède le getter *getRole()* qui permet à Symfony de récupérer lors de la connexion le/les rôle(s) de l'utilisateur. Ces rôles sont stockés sous forme de tableau dans la base de données.

Deux rôles sont définis pour ce projet et sont des rôles attribués aux utilisateurs :

- **ROLE_USER**
- **ROLE_ADMIN**

Symfony propose également des « *status* » qui permettent de préciser certains accès, tel que :

- **PUBLIC_ACCESS**
- **IS_AUTHENTICATED_FULLY**.

Le contrôle des accès

access_control

Les droits d'accès des utilisateurs aux routes du projet est géré au sein du fichier *security.yaml* .

Seul les utilisateurs ayant le rôle *ROLE_ADMIN* peuvent accéder aux pages :

- Liste de toutes les tâches
- liste de tous les utilisateurs

```
34 # Easy way to control access for large sections of your site
35 # Note: Only the *first* access control that matches will be used
36 access_control:
37     - { path: ^/task_list, roles: ROLE_ADMIN }
38     - { path: ^/users_list, roles: ROLE_ADMIN }
39     # - { path: ^/profile, roles: ROLE_USER }
```

Attributes

Exemple @IsGranted

Les Voters peuvent être mise en place dans les Controllers grâce au :

`Symfony\Bundle\FrameworkBundle\Controller\AbstractController::denyAccessUnlessGranted`

```
AdminController.php x
src > Controller > AdminController.php > ...
1 <?php
2
3 // src/Controller/AdminController.php
4
5 namespace App\Controller;
6
7 use App\Repository\TaskRepository;
8 use App\Repository\UserRepository;
9 use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
10 use Symfony\Component\HttpFoundation\Response;
11 use Symfony\Component\Routing\Attribute\Route;
12
13 class AdminController extends AbstractController
14 {
15     #[Route('/admin', name: 'app_admin')]
16     public function index(UserRepository $userRepository, TaskRepository $taskRepository): Response
17     {
18         // Sécurité d'accès
19         $this->denyAccessUnlessGranted('ROLE_ADMIN');
20
21         $users = $userRepository->findAll();
22         $tasks = $taskRepository->findAll();
23
24         return $this->render('admin/index.html.twig', [
25             'users' => $users,
26             'tasks' => $tasks,
27         ]);
28     }
29 }
```

Les attributs peuvent être mise en place dans les Controllers grâce au :
use Symfony\Component\Security\Http\Attribute\IsGranted as AttributeIsGranted;

```
index.html.twig x
templates > home > index.html.twig
1  {% extends 'base.html.twig' %}
2
3  {% block header_title %}
4      <h1>Bienvenue sur Todo List, l'application vous permettant de gérer l'ensemble de vos tâches sans effort !</h1>
5  {% endblock %}
6
7  {% block body %}
8      {% if app.user %}
9          <div class="row">
10             <a href="{{ path('app_task_new') }}" class="btn btn-success">Créer une nouvelle tâche</a>
11             <a href="{{ path('user_task_list') }}" class="btn btn-primary">Consulter la liste des tâches à faire</a>
12             <a href="" class="btn btn-info">Consulter la liste des tâches terminées</a>
13             {% if 'ROLE_ADMIN' in app.user.roles %}
14                 <a href="{{ path('app_admin') }}" class="btn btn-danger">Accès admin</a>
15             {% endif %}
16         </div>
17     {% endif %}
18 {% endblock %}
```

Soit directement au niveau des annotations au sein du moteur de templating Twig.

```
TaskController.php x
src > Controller > TaskController.php > PHP Intelephense > TaskController > index
1  <?php
2
3  namespace App\Controller;
4
5  use App\Entity\Task;
6  use App\Form\TaskForm;
7  use App\Repository\TaskRepository;
8  use Doctrine\ORM\EntityManagerInterface;
9  use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
10 use Symfony\Component\HttpFoundation\Request;
11 use Symfony\Component\HttpFoundation\Response;
12 use Symfony\Component\Routing\Attribute\Route;
13 use Symfony\Component\Security\Http\Attribute\IsGranted as AttributeIsGranted;
14
15 #[Route('/task')]
16 final class TaskController extends AbstractController
17 {
18     #[Route('all/task', name: 'task_list', methods: ['GET'])]
19     #[AttributeIsGranted('ROLE_ADMIN')]
20     public function index(TaskRepository $taskRepository): Response
21     {
22         return $this->render('task/index.html.twig', [
23             'tasks' => $taskRepository->findAll(),
24         ]);
25     }
26 }
```